



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá
Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 2
Laboratorio SOLID
Sebastian Clavijo Tocasuchyl - 1014978141

Punto 1

```
(.venv) PS D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab> pytest tests/test_ex1_srp.py -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab
configfile: pytest.ini
collected 6 items

tests/test_ex1_srp.py::test_car_move_does_not_print PASSED [ 16%]
tests/test_ex1_srp.py::test_car_move_only_updates_position PASSED [ 33%]
tests/test_ex1_srp.py::test_car_move_does_not_touch_disk PASSED [ 50%]
tests/test_ex1_srp.py::test_race_logger_exists PASSED [ 66%]
tests/test_ex1_srp.py::test_race_logger_records_and_saves PASSED [ 83%]
tests/test_ex1_srp.py::test_main_runs_end_to_end PASSED [100%]

===== 6 passed in 0.02s =====
```

¿Por qué es mejor que **Car** no sepa nada sobre logging ni impresión en consola?

Car representa una entidad del dominio responsable exclusivamente de gestionar su posición y movimiento. Si Car maneja impresiones o guardado en archivos, se le otorgan múltiples razones para cambiar: cambios en la física del vehículo o cambios en los requerimientos de la interfaz/console. Separar estas responsabilidades cumple con el SRP y mantiene la clase limpia.

¿Qué se rompería primero si se pidiera guardar resultados en formato JSON en lugar de texto plano?

Si las responsabilidades estuvieran acopladas dentro de Car, tendríamos que modificar la clase Car para cambiar el formato de exportación, arriesgándonos a introducir errores en la simulación de la carrera. Con la responsabilidad aislada en RaceLogger, solo se modifica el método save() de dicha clase sin alterar el comportamiento de ningún vehículo.

Punto 2

```
(.venv) PS D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab> pytest tests/test_ex2_ocp.py -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab
configfile: pytest.ini
collected 4 items

tests/test_ex2_ocp.py::test_car_and_truck_unchanged PASSED [ 25%]
tests/test_ex2_ocp.py::test_motorcycle_and_bicycle_are_vehicles PASSED [ 50%]
tests/test_ex2_ocp.py::test_motorcycle_moves_variably PASSED [ 75%]
tests/test_ex2_ocp.py::test_bicycle_gets_slower_over_time PASSED [100%]

===== 4 passed in 0.01s =====
```

Si tuvieras que editar Vehicle o Track para completar este ejercicio, ¿qué te diría sobre tu diseño?

Indicaría un diseño rígido y mal abstraído. El OCP establece que las clases deben estar *abiertas a la extensión pero cerradas a la modificación*. Si agregar nuevos comportamientos

(como Motorcycle o Bicycle) exigiera modificar Vehicle o Track, significaría que el sistema depende de implementaciones concretas en lugar de aprovechar el polimorfismo mediante el método `move()`.

Punto 3

```
(.venv) PS D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab> pytest tests/test_ex3_lsp.py -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-r
ace-lab\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab
configfile: pytest.ini
collected 3 items

tests/test_ex3_lsp.py::test_unreliable_car_is_a_vehicle PASSED [ 33%]
tests/test_ex3_lsp.py::test_unreliable_car_never_moves_backwards_or_raises PASSED [ 66%]
tests/test_ex3_lsp.py::test_track_can_run_a_full_race_without_crashing PASSED [100%]

===== 3 passed in 0.01s =====
```

¿Qué habrías tenido que agregar a Track si NO hubieras arreglado UnreliableCar?

Se habría tenido que llenar Track con código defensivo, como bloques `try/except` `RuntimeError` para evitar que el programa colapse al fallar un vehículo, y lógica de validación como `max(old_position, new_position)` para evitar que la posición retroceda.

¿Por qué es mejor arreglar la subclase?

Track debe poder confiar en el contrato establecido por la abstracción base Vehicle. Arreglar UnreliableCar garantiza que cualquier subclase pueda sustituir a Vehicle de forma transparente sin que el llamador (Track) deba incluir excepciones particulares para casos defectuosos.

Punto 4

```
(.venv) PS D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab> pytest tests/test_ex4_lsp.py -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-r
ace-lab\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab
configfile: pytest.ini
collected 4 items

tests/test_ex4_lsp.py::test_small_focused_interfaces_exist PASSED [ 25%]
tests/test_ex4_lsp.py::test_gas_car_only_implements_relevant_interfaces PASSED [ 50%]
tests/test_ex4_lsp.py::test_bicycle_only_implements_relevant_interfaces PASSED [ 75%]
tests/test_ex4_lsp.py::test_drone_exists_and_only_implements_relevant_interfaces PASSED [100%]

===== 4 passed in 0.01s =====
```

¿Qué interfaz(es) necesitaría un vehículo futuro como Submarine o Airplane?

- Airplane: Implementaría Movable, Refuelable y Flyable.
- Submarine: Implementaría Movable, Refuelable y una nueva interfaz específica como Swimmable / Divable.

¿Agregar alguno de ellos requiere modificar alguna clase existente? No. Gracias a que las interfaces están segregadas en componentes pequeños (Movable, Flyable, etc.), agregar un nuevo tipo de vehículo o interfaz no altera ni requiere reescribir GasCar, Bicycle o Drone.

Punto 5

```
(.venv) PS D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab> pytest tests/test_ex5_dip.py -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab
configfile: pytest.ini
collected 4 items

tests/test_ex5_dip.py::test_race_constructor_accepts_racers_from_outside PASSED [ 25%]
tests/test_ex5_dip.py::test_race_does_not_hardcode_its_roster PASSED [ 50%]
tests/test_ex5_dip.py::test_race_runs_with_an_arbitrary_roster PASSED [ 75%]
tests/test_ex5_dip.py::test_race_works_with_two_different_rosters_unmodified PASSED [100%]

===== 4 passed in 0.01s =====
```

¿Cómo el hecho de inyectar el *roster* hace que Race sea más fácil de probar con pruebas unitarias sin la animación de la terminal?

Al pasar los competidores por el constructor `Race( racers )`, se pueden inyectar objetos falsos (*test doubles* o *mocks*) en las pruebas unitarias. Esto permite verificar la lógica de la carrera de forma determinista y rápida con `pytest`, sin depender de la ejecución real de clases de bajo nivel ni de rendering gráfico en terminal.

WRAP UP

- Test Completo

```
(.venv) PS D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab> pytest -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\UNAL\Sexto Semestre\IngeSoft2\solid-race-lab-solution\solid-race-lab
configfile: pytest.ini
testpaths: tests
collected 21 items

tests/test_ex1_srp.py::test_car_move_does_not_print PASSED [ 4%]
tests/test_ex1_srp.py::test_car_move_only_updates_position PASSED [ 9%]
tests/test_ex1_srp.py::test_car_move_does_not_touch_disk PASSED [ 14%]
tests/test_ex1_srp.py::test_race_logger_exists PASSED [ 19%]
tests/test_ex1_srp.py::test_race_logger_records_and_saves PASSED [ 23%]
tests/test_ex1_srp.py::test_main_runs_end_to_end PASSED [ 28%]
tests/test_ex2_oci.py::test_car_and_truck_unchanged PASSED [ 33%]
tests/test_ex2_oci.py::test_motorcycle_and_bicycle_are_vehicles PASSED [ 38%]
tests/test_ex2_oci.py::test_motorcycle_moves_variably PASSED [ 42%]
tests/test_ex2_oci.py::test_bicycle_gets_slower_over_time PASSED [ 47%]
tests/test_ex3_lsp.py::test_unreliable_car_is_a_vehicle PASSED [ 52%]
tests/test_ex3_lsp.py::test_unreliable_car_never_moves_backwards_or_raises PASSED [ 57%]
tests/test_ex3_lsp.py::test_track_can_run_a_full_race_without_crashing PASSED [ 61%]
tests/test_ex4_isp.py::test_small_focused_interfaces_exist PASSED [ 66%]
tests/test_ex4_isp.py::test_gas_car_only_implements_relevant_interfaces PASSED [ 71%]
tests/test_ex4_isp.py::test_bicycle_only_implements_relevant_interfaces PASSED [ 76%]
tests/test_ex4_isp.py::test_drone_exists_and_only_implements_relevant_interfaces PASSED [ 80%]
tests/test_ex5_dip.py::test_race_constructor_accepts_racers_from_outside PASSED [ 85%]
tests/test_ex5_dip.py::test_race_does_not_hardcode_its_roster PASSED [ 90%]
tests/test_ex5_dip.py::test_race_runs_with_an_arbitrary_roster PASSED [ 95%]
tests/test_ex5_dip.py::test_race_works_with_two_different_rosters_unmodified PASSED [100%]

===== 21 passed in 0.04s =====
```

- Pregunta de Cierre (Wrap-up): Ejemplo de Proyecto Real:

Proyecto: Un e-commerce para procesar compras online.

El Problema (Violación de DIP y SRP): La clase `OrderService` creaba directamente a `PayPalPayment()` por dentro para cobrar, y de paso enviaba el correo de confirmación y generaba el PDF de la factura.

Lo que se rompió: Cuando se necesitó agregar **Stripe**, tocó modificar el código de `OrderService` metiendo condicionales `if/else`. Además, no se podían hacer pruebas unitarias sin intentar hacer cobros reales a la API de PayPal.

La Solución SOLID: Inyectar una interfaz genérica `PaymentProcessor` en el constructor de `OrderService` (DIP) y pasar la generación de facturas y correos a clases independientes (SRP).